

UNITY IN LEARNING • VITYARTHI PROJECT

FRAMEWORK

Project: Smart Campus Resource & Study Space Reservation System

Course: Python Essentials (VITyarthi)

Domain: Core Software Systems / Database & Application Design

Repo: GitHub modular implementation (unit tests included)

Doc Version: 1.0 (Final) | **Completion:** Academic Year

Declaration: *The report and the attached codebase are my original academic work, created by Astik Mishra (Reg. No. 26BCE10706), B.Tech CSE, following the VITyarthi flipped-course evaluation rules for Python Essentials (VITyarthi).*

Submitted By: Astik Mishra | **Reg. No.:** 26BCE10706 | **Program:** B.Tech, Computer Science & Engineering (CSE)

1. Introduction

Campuses don't run on "one type of room" anymore. You've got silent pods, loud discussion rooms, HPC lab benches, seminar halls with finicky AV gear—the works. And as enrollment climbs, the same spaces get chased by more people. Chaos follows.

This Smart Campus Resource & Study Space Reservation System is built to keep that chaos in check. Light. Dependable. Secure. No more paper sign-up sheets taped to doors. No more scattered WhatsApp messages that mysteriously "count as a booking." A single booking engine sits in the middle, spreads access more fairly, blocks time-slot collisions, and gives admins a live view of what's happening.

2. Problem Statement

Old-school campus facility booking has a bunch of recurring problems. The kind that wastes time fast:

- **Double-booking clashes:** Manual requests collide. Two groups show up. Nobody's happy. Workshops get derailed.
- **Hoarding and dead-time lockouts:** If you let people reserve forever, they will. Rooms sit idle, but "booked."
- **No real audit trail:** Paper logs don't tell you usage patterns. Peak hours? Popular rooms? You're guessing.
- **Weak role separation:** Basic setups can't cleanly separate student actions from admin overrides. That's a security mess.

3. Functional Requirements

Per the VITyarthi project checklist, the system is split into three main functional modules.

Module 1 — User & Access Management

What it does:

- Secure registration plus PBKDF2-based authentication.
- Role-Based Access Control (Student vs Admin).
- Session state handling.

I/O Dynamics: **Inputs:** username, email, password → **Outputs:** session token, role claims

Module 2 — Booking & Conflict Engine

What it does:

- Browse resources in real time.
- Stop overlaps using SQL interval checks.
- Validate duration limits and lead-time rules.

I/O Dynamics: **Inputs:** resource ID, start/end time → **Outputs:** confirmation ticket or a conflict error

Module 3 — Analytics & CSV Reporting

What it does:

- Admin-facing operational dashboard signals.
- Utilization and peak-usage metrics.
- One-click CSV export for audit logs.

I/O Dynamics: **Input:** admin export command → **Output:** summary metrics, CSV audit file

4. Non-Functional Requirements

The project targets a few quality bars that matter in real use:

- **Performance:** Conflict checks should feel instant (< 50 ms).
Mechanism: Indexed SQLite queries on (resource_id, start_time, end_time).
- **Security:** No plaintext passwords. Injection resistance.
Mechanism: PBKDF2 SHA-256 with per-user salt; parameterized SQL statements.
- **Reliability:** ACID compliance and no race-condition double bookings.
Mechanism: SQLite WAL mode plus explicit transactions and rollbacks on integrity failures.

6. System Design Diagrams

(Figure 2: Use Case Diagram — Student and Admin interactions)

(Figure 3: ER Diagram — normalized database schema mapping Users, Resources, and Reservations)

7. Key Design Decisions & Rationale

Some choices were easy. Some took arguing with myself.

- **SQLite as the data engine:** It's embedded, requires almost zero setup, and still gives atomic transactions with standard SQL. Perfect for an academic build that still behaves like a “real” system.
- **Standard-library cryptography:** Using `hashlib.pbkdf2_hmac` avoids dependency sprawl while staying on proven security ground.
- **Deterministic overlap logic in SQL:** The overlap rule lives where it should: near the data. The predicate `(start < end) AND (end > start)` is clean, mathematical, and doesn't get “creative” under load.

8. Implementation Details

The codebase is split across focused files under `src/`, `data/`, and `tests/`.

- `src/config.py` — global config and paths (Key: `DB_PATH`, `MAX_BOOKING_HOURS`)
- `src/exceptions.py` — custom domain exceptions (Key: `SlotOverlapError`)
- `src/database.py` — connection handling and schema setup
- `src/models.py` — typed dataclasses (Key: `User`, `Resource`, `Reservation`)
- `src/auth.py` — PBKDF auth flow (Key: `AuthService.authenticate_user()`)
- `src/booking_engine.py` — core reservation rules and conflict logic
- `src/analytics.py` — metrics and CSV exporter
- `main.py` — interactive CLI coordinator (Key: `CampusReservationApp.main_menu()`)

Core conflict-checking snippet (Booking Engine):

```
def check_conflict(self, resource_id: int, start_iso: str, end_iso: str) ->
bool: query = ''' SELECT COUNT(*) FROM reservations WHERE resource_id = ? AND
status = 'CONFIRMED' AND (start_time < ? AND end_time > ?) ''' with
self.db.get_connection() as conn: cursor = conn.cursor() cursor.execute(query,
(resource_id, end_iso, start_iso)) return cursor.fetchone()[0] > 0
```

9. Screenshots & Experimental Results

The terminal UI keeps things blunt and readable. Menus are clear. Errors are explicit. Active bookings show up in neat tables.

```
===== SMART CAMPUS
RESOURCE & STUDY SPACE RESERVATION SYSTEM
===== ID | Name | Category
| Cap | Location ----- 1 |
VIT Central Library - Silent Study Pod | study_room | 2 | Central Library L2 2 | VIT
Central Library - Group Discussion B | study_room | 6 | Central Library L2 3 | SJT High
Performance Computing Bench 04 | lab_bench | 1 | SJT 4th Lab [+] SUCCESS! Booking
confirmed. Reservation ID: #4489 Start Time: 2024-09-XX 12:00 | End Time: 2024-09-XX
14:00
```

10. Testing Approach & Verification

Testing isn't optional here. The project includes an automated suite in `tests/test_runner.py` built with Python's `unittest`, using temporary isolated DB fixtures so tests don't contaminate each other.

Key cases verified:

- `test_user_registration_and_auth` — hashes passwords, rejects bad credentials.
`PASSED [0.12s]`
- `test_time_slot_validation` — blocks >4-hour bookings and anything in the past.
`PASSED [0.08s]`
- `test_reservation_creation_and_overlap` — refuses intersecting time slots.
`PASSED [0.15s]`
- `test_reservation_cancellation` — users can only cancel their own reservations.
`PASSED [0.09s]`

11. Challenges Faced & Mitigations

A couple issues showed up that look small... until they bite.

- **Timezone / datetime consistency:** Raw strings can betray you when formats drift.
Fix: Force ISO-8601 normalization everywhere (YYYY-MM-DDTHH:MM:SS) and keep queries consistent.
- **Overlap edge cases:** The “14:00 ends, 14:00 starts” scenario is a classic trap.
Fix: Strict `<` and `>` in the overlap predicate so adjacent slots don't get flagged as conflicts.

12. Learnings & Key Takeaways

This project, built under the VITyarthi flipped model, hammered in a few practical lessons:

- **RBAC isn't just "a feature":** It shapes the whole flow of the application.
- **PBKDF hashing with salts is non-negotiable:** Essential if you care about basic password safety.
- **SQLite is serious:** When you treat transactions, indexes, and constraints properly, it performs exceptionally well.
- **Unit tests with isolated fixtures:** These save you from regression whiplash during iterative development.

13. Future Enhancements

Plenty of room to push this further:

- **Web + mobile UI:** A React + FastAPI dashboard with drag-and-drop scheduling would make adoption way easier than a CLI.
- **IoT door-lock integration:** ESP/RFID/QR-based unlock flow for pods and rooms. Book it, scan it, enter.
- **Email/SMS alerts:** Automated reminders 15 minutes before start time, and potential cancellation warnings.

14. Academic References

1. VITyarthi Project Development & Submission Guidelines, Academic Year 2023–2024.
2. Python Standard Library Documentation — `sqlite3`, `hashlib`, `unittest`. Python Software Foundation.
3. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database System Concepts* (7th ed.). McGraw-Hill Education.
4. Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.